Parametric Computer Aided Design of Crochet Patterns

William Gooch

University of
St Andrews

Supervisor: Jason Jacques

Date of Submission: 17th January 2025

Abstract

Currently, design of crochet patterns requires significant trial and error, where each attempt and modification requires time and materials to test. It would be helpful for crocheters to be able to design and visualise patterns before trying to make them, and would help to reduce iteration cost. Additionally, patterns published online will often list multiple versions for different sizes, with only minor differences in shape between them. Each of these versions also needs to be tested in much the same way, further increasing cost and time investment.

This dissertation proposes Polyhook, a software package built for the design and visualisation of crochet patterns. This includes a data format that can be used to represent any generic crochet pattern, based on a procedural scripting language. The software supports rendering of the created pattern as a three-dimensional model so a user can inspect that the pattern is shaped correctly, and a way to parametrise these patterns so they can scale to different sizes with only one distributed file. It also supports visual scripting with a limited subset of features for faster iteration of small crochet parts.

Declaration

I declare that the material submitted for assessment is my own work except where credit is explicitly given to others by citation or acknowledgement. This work was performed during the current academic year except where otherwise stated.

The main text of this project report is 11,729 words long, including project specification and plan.

In submitting this project report to the University of St Andrews, I give permission for it to be made available for use in accordance with the regulations of the University Library. I also give permission for the title and abstract to be published and for copies of the report to be made and supplied at cost to any bona fide library or research worker, and to be made available on the World Wide Web. I retain the copyright in this work.

Contents

Chapter 1

Introduction

1.1 Crochet and Amigurumi

Crochet is a form of manual yarn work, related to knitting. Unlike knitting, it utilizes a single hook rather than two needles. A crocheted work is composed of either *rows* or *rounds*, where a row spans from one end to another and a round joins to itself at the start. Each row or round is composed of many *stitches* in sequence, where each stitch works into the row or round below it. Figure 1.1 shows the parts of a crocheted work, including the parts of a stitch.

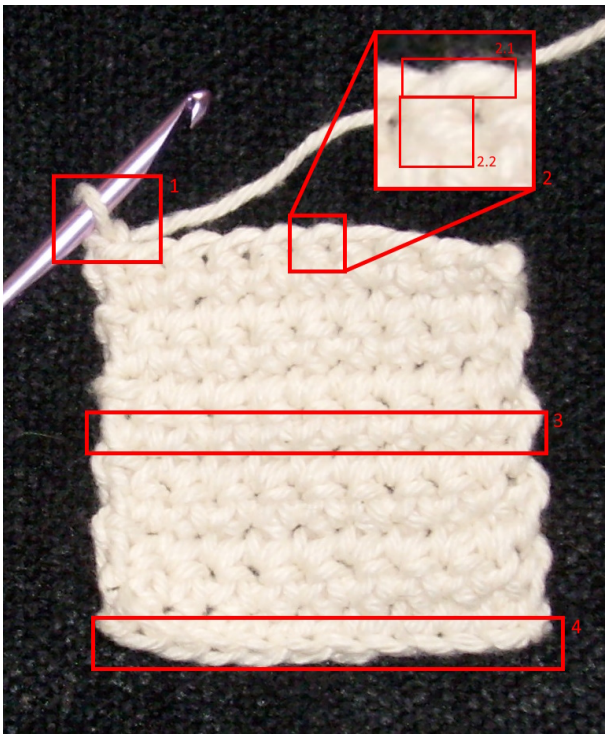


Figure 1.1: An annotated image of a crochet square.[1]

#	Description
1	The stitch currently being worked, with the hook through the current working yarn.
2	A 'double crochet' stitch.
2.1	The 'front loop' of the stitch.
2.2	The 'post' of the stitch.
3	A row of double crochet stitches.
4	The 'foundation row' of the square, the first row worked, made entirely of chain stitches.

The most important stitches in crochet are 'chains' and 'double crochet'. A chain adds height to a row, and is (often) used to build the foundation row of a flat work. A double crochet works into another stitch, usually one on the previous row, and has a height equivalent to one chain stitch. Other more complex stitches exist, such as trebles, half doubles, slip stitches and picots.

Crochet is distinct from knitting in that it is able to easily produce works that form complex three-dimensional shapes. Typically, to make a flat piece, a crocheter would work one stitch for each stitch in the previous row, however working more or fewer stitches into each base row stitch can cause the piece

[1]Image from Wikimedia Commons: `https://commons.wikimedia.org/wiki/File:Singleturning.jpg` licensed under Creative Commons 4.0 BY-SA `https://creativecommons.org/licenses/by-sa/4.0/`

to curve - these are called *increases* and *decreases*. The exact shape can be modified by changing the number of increases or decreases in each row.

Amigurumi is a Japanese style of crochet that uses this property of the craft to create three-dimensional stuffed toys and other sculptures, which can be simple or very complex, with many different parts sewn together.

1.2 Project Outline

For someone wanting to make a crochet pattern, the process can very often rely on a lot of trial and error. Crochet is a very sequential craft, meaning that it's hard to make amendments to a section that has already been made without unraveling the whole work up to that point. Each attempt at finding the right shape therefore either takes a lot of time for each stitch to be undone and then remade, or a lot of material cost for a second test piece to be attempted.

As well as this, many crochet patterns for garments will need to vary the size of the work to match the intended size of the garment. This can increase the difficulty of writing a pattern significantly, since writing a single pattern now must involve writing multiple, even if they only have small changes between them. If the pattern is to be published online, typically the crafter will want to test each size to make sure their pattern is accurate, thereby also increasing the material or time investment required.

In related crafts like knitting, computer-aided design and digital fabrication have facilitated rapid design and pattern development, and so it's reasonable to imagine that a digital approach can also help with crochet, despite the difficulty of automated manufacturing, explored in Section 2.1.2. This dissertation proposes Polyhook, a software solution that allows a crocheter to visualize the shape and size of a crocheted work before they make it. A user can adapt a pattern into Polyhook's pattern format, built on procedural scripting, and render the resulting work as a 3D-model. Additionally, the use of a scripting language here allows for dynamic sizing of pieces based on input parameters, which can be then exposed to another user to modify for their use case, solving the garment sizing problem explained above.

Chapter 2

Context Survey

2.1 Crochet in Computing

2.1.1 Crochet Patterns

There are a couple popular formats for distribution of crochet patterns, chief among which is just a list of instructions, with or without accompanying images. Typically, this will be written in rows or rounds, where each row or round will list the stitches within it, typically with abbreviations to make the pattern more concise.

The other common option is a crochet chart, which uses standardized symbols to represent stitches and where they connect. These symbols are directional, to show where they should be inserted into. However, typically these are only easily usable for flat pieces, as the nature of projecting a three-dimensional shape into two dimensions would cause overlap between stitches and would make the chart look confusing.

```
row 1: chain 18

row 2: tr in 4th chain from hook, ch 1, tr in
       same stitch, skip 2 ch, (tr, ch 1, tr
       in same stitch, skip 2 ch) repeat to
       end, tr in last ch

row 3: ch 3, (tr in next ch, ch 1, tr in same
       stitch) repeat to end, tr in last ch

row 4+: repeat row 3
```

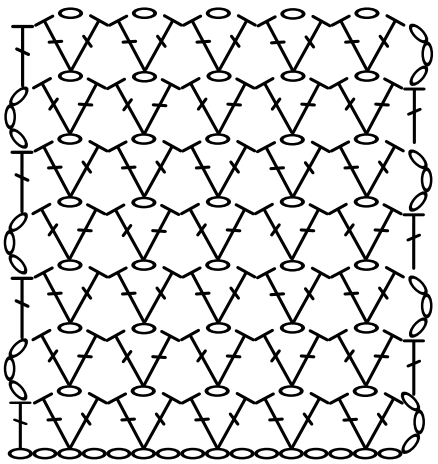


Figure 2.1: A set of crochet instructions vs. a crochet chart

Due to differences in standardisation[1], patterns written in the UK use different terms for stitches than patterns written in the US - particularly, UK terms refer to the number of loops on the hook during the beginning of the stitch[2], while US terms refer to the height added to the row by working it[3]. This slight difference means that UK terms are typically one 'higher' than US terms, for example what the US refers to as a *single crochet*, the UK refers to as a *double crochet*. For the sake of clarity, all terminology for the remainder of this report will be stated in **UK format**.

2.1.2 Crochet Machines

Historically, the development of crochet machines has been very slow. This is unlike knitting, where commercial knitting machines are widely available and very efficient. This is in part due to the fact that knitting typically has only one possible insertion point for each stitch, and that each stitch only depends on the previous row, rather than the previous stitch in the same row, meaning a whole row can be knitted at once. In contrast, crochet has many different possible insertion points depending on the stitch being performed - an increase will insert into the same point as the previous stitch, whereas a decrease will insert into two stitches before being completed. Additionally, each crochet stitch requires the previous stitch to be completed before starting. Since the insertion point can change, a crochet machine would need to have awareness of the position of each possible insertion point, and thus would need very precise sensors for each stitch in the piece.[4]

However, there have been some developments. Many commercial 'crochet machines' don't perform crochet and instead perform specialized forms of knitting designed to look like crochet, for example *warp knitting*. There are some rudimentary machines, however - for example, a patent has been submitted for a machine that can crochet small rectangular pieces with double crochets[5], however it is limited by the above restrictions and cannot use anything but the most recent insertion point, and can't change the number of stitches mid-piece. A more modern development was Croche-Matic[6], which is able to crochet in rounds to create spherical models, working increases and decreases as necessary, although it is somewhat inconsistent having a general success rate for each stitch of around 50.7%.

The lack of crochet machinery means that rapid prototyping is difficult, because every attempt at a pattern must be made by hand. This makes development of new patterns a slow process of trial and error, made only easier by the heuristics that experienced crocheters can follow. For less seasoned crocheters, however, that intuition is harder to build up, so trying to make something unique often leads to wasted materials, and wasted time.

2.1.3 Crochet Software

Some solutions currently do exist for creating crochet charts - for example, the website Stitch Fiddle[1] supports this as well as other crafts like knitting, cross stitch, etc. However, it is more of a graphical editor than anything, allowing one to place stitch symbols in a mostly free-form way. While this is very flexible, it keeps no information as to the structure of the piece, and so would be unable to handle automated generation of charts or models. Knitting software, on the other hand, is much more prevalent, with options like KnitBird[2] and Stitchmastery[3] available for hand-crafting patterns, and DesignaKnit[4] for computerised knitting machines.

Generally, though, the process of creating crochet patterns involves experimentation and writing out instructions in a list manually. This requires a crafter to note down the stitches as they do them, and requires a lot of trial and error to find the shape they want. It could be helpful to have a piece of software model the shape of a crocheted item so that it can be previewed before crafting it.

Crochet patterns are often shared and sold on websites like Ravelry or Etsy. However, often these distributed patterns have differences in formatting, terminology and approach, as idiosyncrasies of the designer themselves. Additionally, if a pattern includes multiple design options, for example for different sized garments, then the pattern needs to be duplicated for each potential size, and the designer needs to test out each possible size to make sure the shape is as they expect.

2.2 Parametric Modelling

Parametric modelling is a paradigm of 3D modelling that uses sequential operations and constraints to construct a 3D object. This is in contrast to direct modelling, which uses direct manipulation of surfaces or vertices to create a model. Examples of parametric modelling software include:

- **FreeCAD** *(free, open-source)* `https://www.freecad.org/`
  A CAD tool that works by creating 2D sketches from constraints and extruding them into 3D.
- **Autodesk Fusion 360** *(proprietary)* `https://www.autodesk.com/products/fusion-360/overview`
  A similar tool with a larger feature-base of operations and constraints.
- **OpenSCAD** *(free, open-source)* `https://openscad.org/`
  A tool that allows users to create models programmatically, using a custom imperative scripting language.
- **CadQuery** *(free, open-source)* `https://cadquery.readthedocs.io/en/latest/`
  A Python module similar to OpenSCAD using the Open CASCADE library to generate geometry with a fluent API.

The programming-based options above have the benefit of specifying the shape of the object with extreme precision, however they lack the usability of GUI-based CAD software. Either way, one of the key features

[1] `https://www.stitchfiddle.com/en/company/about`
[2] `https://knitbird.com/`
[3] `https://stitchmastery.com/`
[4] `https://softbyte.co.uk/knitwearpatterndesignsoftwareprogramdesignaknit.htm`

of parametric CAD is propagation, whether earlier operations in the sequence can be modified, and said changes propagate to later steps in the process. However, these processes aren't intuitive, and often rely on heuristics that aren't clear or known to the user in advance to resolve conflicts[7].

Another benefit of parametric CAD is adaptability. Often, models are shared on websites like Thingiverse for the purpose of 3D printing, and it can be easier to distribute the raw CAD files (for example in OpenSCAD format) so that the end user can adjust dimensions for their own applications. Two of the 20 most popular models on Thingiverse [5] use this technique to allow users to create parts to exactly their specification. This approach could be beneficial to alleviate the aforementioned sizing problem with many crochet patterns.

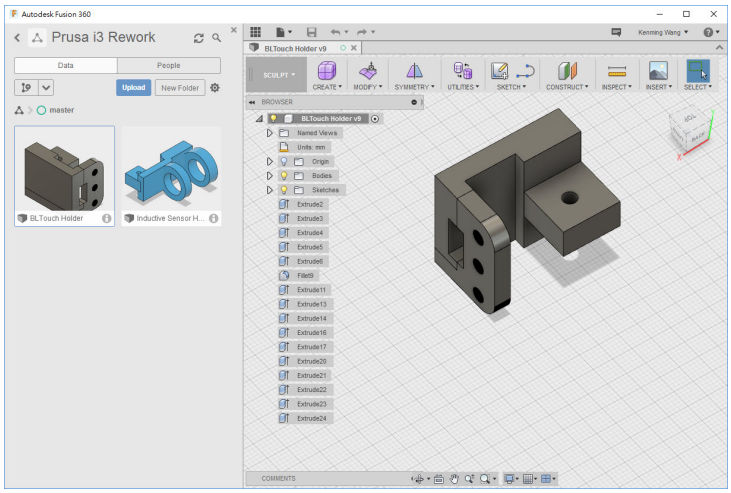


Figure 2.2: The process of modelling an example object in the parametric CAD tool Autodesk Fusion 360.[6]

Given crochet's nature as a sequence of stitches, it's relatively easy to envision a parametric approach working well for computer-aided crochet pattern design, however most parametric CAD techniques revolve around solid geometry with high amounts of precision, so many common tools such as Boolean operations, fillets, bevels and sweeps may not suit this application. Therefore, it's important to determine a set of elementary operations for the specific case of crochet design.

2.3 Related Works

In *Language and tool support for 3D crochet patterns*, Seitz et al.[4] discuss possible solutions for digital representation of generic crochet patterns, and develop a tool to create three-dimensional charts. This has a slightly different focus from this project, as it targets those who are already familiar with crochet and revolves around an editor that creates crochet charts and involves placing each stitch individually. This project, on the other hand, focuses on creating a more realistic 3D model of the pattern, and generating these stitches automatically through a more user-friendly process of parametric operations.

That said, though, Seitz et al. do propose a data structure that could prove to be extremely useful in terms of representing a crochet pattern - a graph consisting of nodes representing stitches and edges representing relationships between them. Primarily, there would be two kinds of nodes:

- `stitch`: represents a single stitch in the crochet graph.
- `chain space`: represents any other insertion point, for example within a loop of chains that form a space in the middle.

and four kinds of edges:

[5] Nut Job: `https://www.thingiverse.com/thing:193647` and Parametric Hinge `https://www.thingiverse.com/thing:2187167`

[6] Image by Kenming Wang `https://www.flickr.com/photos/kenming_wang/32276624594`, licensed under Creative Commons BY-SA 2.0 `https://creativecommons.org/licenses/by-sa/2.0/`

- `previous` relationship: the previous stitch in the same row that this stitch should connect to - one-to-one.
- `inserts` relationship: any nodes that this stitch should insert into - may be multiple in the case of decreases, or may be shared across stitches in the case of increases - many-to-many
- `neighbours` relationship: specifically for `chain space` nodes, represents the surrounding stitches that make up the chain space.
- `slip stitch` relationship: a slip stitch is a special kind of stitch that just joins two stitches together directly without creating a new node in between them.

This structure fits the domain very well and allows for a lot of flexibility in the kinds of pattern one can model.

Another idea Seitz et al. propose is the use of graph relaxation to create crochet charts. In the case of this project, the primary intent is not to create crochet charts, but the core concept remains useful - simply specify the connections between nodes, and let a graph relaxation algorithm find the model that represents it best.

Another related work is AmiGo by Edelstein et al.[8] which proposes an automated algorithm for generating a crochet pattern from a generic 3D model - essentially the inverse of my own project. While much of the content isn't entirely relevant to this project, some parts stand out as potentially useful - particularly, they refer to crochet patterns as being akin to a set of assembly instructions, and thus propose using loop unrolling and other such reconstruction techniques to turn a crochet 'execution trace' - that is, the final list of stitches in the model, into a usable pattern. This brings to mind some possibilities of refactoring patterns into more simplified versions in order to make them easier to read when exporting to particular formats.

Chapter 3

Requirements Specification

3.1 Objectives

The set of objectives set out at the beginning of this project in the DOER document were:

- Primary objectives:
  - Develop a standardised data format to represent crochet patterns.
    - Include support for standard crochet stitches: chains, magic rings, slip stitches, double and treble crochet, increases and decreases.
    - Allow users to define their own unique stitches and extend the format.
  - Create software to allow users to develop crochet patterns in a parametric fashion, by composing basic operations in sequence with repetition and subroutines.
  - Render wireframe models of the expected result of the pattern, involving simulation of the expected curvature and shaping of the work.
- Secondary objectives:
  - More sophisticated rendering of the finished article, involving realistic rendering with texturing and lighting.
  - Allow users to compose multiple distinct crocheted parts by sewing, for example join a head and body section together.
  - Render patterns to other formats, such as crochet charts.
- Tertiary objectives:
  - Support other crochet stitches: front and back loop stitches, extended double, half treble, spike stitches.
  - Implement a standard library of stitches composed from other basic stitches, e.g. picot, crocodile stitch, shell stitch, popcorns.
  - Instructional mode for new crocheters, with a step-by-step walk-through of any given pattern.

3.2 Evaluation Criteria

In addition to the objectives set out above, there are more subjective criteria that the final product will be evaluated against.

The software should be *accurate*, meaning the rendered model generated from a pattern should be similar to the physical product created by following the pattern with real crochet materials. This could be tested in two ways: adapting existing crochet patterns into the program and comparing with the real product, and creating a pattern in software and following that pattern to produce a real article. To help with speed of iteration, the former approach is the one used here.

The software should also be *flexible*, meaning it should be capable of producing a wide range of patterns. This can be checked by ensuring the patterns used in development and testing are all distinct, and demonstrate many different features of pattern creation.

Finally, the software should be *usable*, by two particular user types:

- A *pattern creator*, who can develop their own patterns and distribute them for others to use, should have understanding of crochet fundamentals, and parametric CAD – particularly a code-based package like OpenSCAD or CadQuery. A more visual approach would also be helpful to assist in rapid prototyping of ideas, although the feature set for this can be more restrictive.
- A *pattern user*, who can download a pattern created by the above and modify its parameters to suit their needs, should be able to use the software with limited technical knowledge or training.

This means standard design practices should be followed to make the process as smooth for both kinds of users as possible. Additionally, this usability criterion requires the program to be decently performant. The application should never crash or freeze, even if the pattern is very large, and a reasonably-sized model should be generated in a reasonable time. Here I take 2500 stitches as reasonably-sized - while this may be too low for garments or lacework, for the application of Amigurumi this number is high enough to include most patterns, particularly those on the smaller side.

3.3 Requirements

Given the objectives set out, and the more general evaluation criteria, I developed the following set of requirements:

1. **Creation** – A *pattern creator* should be able to:
   - 1.1. write program code representing a crochet pattern, with standard crochet instructions (double crochet, chain, increase, decrease, etc.), repeated shorthand for multiple instructions, and subroutines to define custom stitches
   - 1.2. define parameters for the end user to modify
   - 1.3. sew multiple distince crocheted parts together
   - 1.4. implement each of these tasks in a simple and readable manner
   - 1.5. use a visual system to create a crochet pattern, with standard crochet instructions, as above, and repetition blocks for multiple instructions. Visual patterns should be faster to create, and can therefore be simpler than textual scripts
   - 1.6. save and distribute the pattern as a file to other users
2. **Usage** – A *pattern user* should be able to:
   - 2.1. open a pattern file created by a *pattern creator*
   - 2.2. modify the parameters defined by the pattern creator without editing the pattern itself
   - 2.3. export the list of instructions to craft the real article
   - 2.4. perform all these tasks with no technical or programming knowledge
3. **Visualization** – Any user should be able to:
   - 3.1. visualize the currently opened pattern as a three-dimensional model, which should:
     - 3.1.1. resemble the real article created by following the instructions in the pattern
     - 3.1.2. not have any unnecessary curvature or folding, even if the real article can fold or bend
     - 3.1.3. if three-dimensional and enclosed, inflate as though stuffed with material
   - 3.2. restrict the model to two-dimensions, in the case of a flat piece like a doily or shawl
   - 3.3. move the view around the model using the mouse to show all angles
   - 3.4. zoom in to the model using the scroll wheel
4. **Flexibility** – The pattern creation system should:
   - 4.1. allow for most patterns to be represented using the standard operations
   - 4.2. allow fine control over the pattern, if standard operations don't suit their needs
5. **Usability** – The application should:
   - 5.1. follow standard UI design practices where possible
   - 5.2. provide sensible error messages to the user
   - 5.3. take no more than 30 seconds to render a pattern with at most 2500 nodes
   - 5.4. run at a consistent frame rate and never freeze

Chapter 4

Software Engineering Process

The methodology used for the project was an informal agile system. Design, implementation and testing of features were performed concurrently and in short iterative cycles. That said, the method was informal and did not use any traditional task management system like kanban, since it required no collaboration or stakeholder visibility. Progress was instead checked against the project objectives and requirements, and weekly meetings with the supervisor were taken to check up on progress and occasionally demo new features.

Generally, features were split into three categories:

- Pattern representation, model generation and optimisation.
- Language design and pattern creation
- Rendering

Rendering was the first focus, as a basic wireframe renderer gave a way to test and demonstrate the model generation, and later the pattern creation.

4.1 Language Choice

When deciding on a language to use for the project, the main priorities were efficiency (Req. 5.3.), library support and graphics support. Additionally, familiarity with the language was a significant factor, since fast iteration was desired and learning a new language or paradigm could slow down development.

With this in mind, Rust was the language chosen for a few reasons:

- Rust is highly efficient, and gives a lot of fine-grained control over memory management in order to help to build efficient algorithms.
- Rust is memory-safe, meaning that even with a lower-level granularity there is no risk of undefined behaviour.
- Rust's package ecosystem is wide and well-documented, and dependency management is seamless. Namely, Rust has packages for:
  - Graph representation (`petgraph` `https://docs.rs/petgraph/latest/petgraph/`)
  - Linear algebra (`glam` `https://docs.rs/glam/latest/glam/`)
  - Graphics (`wgpu` `https://docs.rs/wgpu/latest/wgpu/`)
  - UI (`egui` `https://docs.rs/egui/latest/egui/`)
- Bindings for many different graphics APIs, including modern ones such as Vulkan or WebGPU.

4.2 Testing Process

Rust has in-built unit testing support using the `#[test]` macro and `cargo test` command. While the software was not developed in a test-driven manner, unit tests were written alongside code to ensure it worked as expected.

Generally speaking, it's hard to test the rendered results in an automated manner because the accuracy criterion (Req. 3.1.1.) is largely visual and somewhat subjective. Therefore, evaluation of this requirement was done manually rather than using unit testing.

Chapter 5

Ethics

This project focuses more on a proof-of-concept for an interchange format based on program code than a user-facing application. Because of this, my testing doesn't involve any user studies or usability testing, and so there weren't any ethical concerns to raise.

The no-issues form is available in Appendix A.

Chapter 6

Design

6.1 Core Structure

The application is designed as a pipeline of five stages, in summary:

1. **Visual Scripting** - allows a user to sequence operations visually to produce a pattern. (Req. 1.5.) Transpiles to a text script for the next stage.
2. **Textual Scripting** - allows a user to write program code to sequence stitches using the Rhai embedded scripting language. (Req. 1.1.)
   (a) **Parametrization** - a post-processing step where exported variables can be modified by the end user of the pattern. (Req. 2.2.)
3. **Pattern** - the internal graph representation produced by the script, where stitches are nodes connected by edges representing their relationships.
4. **Graph Layout** - this stage gives a position to every node in the graph, using the algorithm explained in Section 7.2. (Req. 3.1.)
5. **Polygon Generation** - this stage converts each stitch node to a polygon, depending on its connections to other nodes.
6. **Rendering** - the final output to the screen, given the model generated in stage 5.

This structure meant that, as long as I had the input and output data format for each stage defined, they could mostly be developed in isolation, helping to reduce tight coupling in the final application.

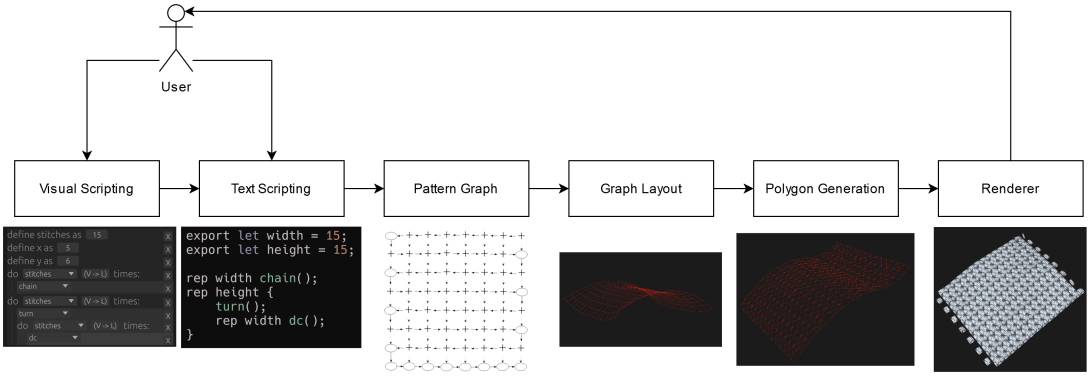


Figure 6.1: The pipeline structure of the application.

6.2 Visual Scripting

The process for an end user either starts with visual scripting or textual scripting. Visual scripting is a more simplistic method of generating basic models than textual scripting (Req. 1.5.), and as such the feature set is smaller than the features afforded by the latter.

The visual scripting system in the final product supports creating stitches, repetition and variable definition. This is a small feature set, but more complex tasks and patterns are better suited to textual scripting, even if it more complex to use. Visual scripting is structured in expressions which can be sequenced together. Steps can be added anywhere, including nested repetitions.

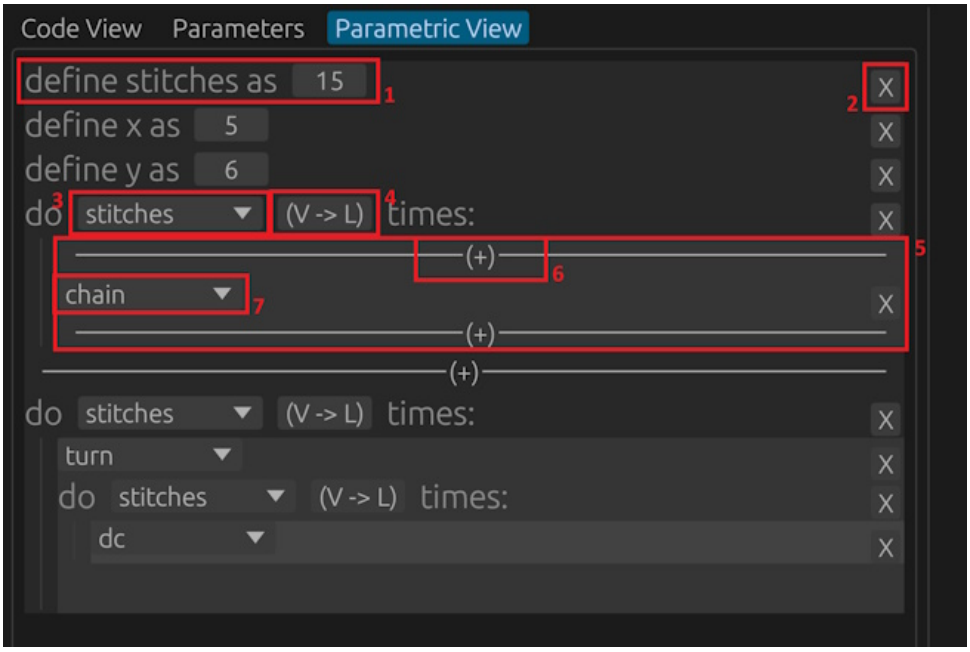


Figure 6.2: An annotated view of the visual editor.

#	Description
1	Variable definition statement.
2	Deletes a step from the sequence.
3	Selects a variable to use for the number of repetitions.
4	Toggles a repeat statement to use a literal value or variable.
5	The body of the repeat statement.
6	Adds a step at this position. Only appears while a surrounding step is hovered.
7	A function call, such as one that allows you to add a stitch.

This transpiles down to the textual scripting language used for the next layer, as the textual script is the canonical representation of a pattern. This also means that the visual scripting can be used to prototype smaller parts, then the resulting code can be embedded in a larger pattern.

6.3 Textual Scripting

When designing the textual scripting system, there were a few things to consider:

- Ease of converting a pattern in written format to a script (Req. 1.4.)
- Ease of translating between visual scripting and textual scripting.
- Flexibility to parametrize scripts based on constants (Req. 2.2.)

Given this, there were two main options. One that was considered is using a Lisp-like paradigm, where program code and data structure are one and the same. This means that parsing and reducing the syntax tree of a script is equivalent to evaluating the script. This provides an easy way of supplying a list of instructions to the next stage, creating the pattern graph. However, Lisp's status as a functional language makes it hard to use for such a procedural process as crochet. It's also unintuitive for people not familiar with functional programming, and it could therefore be hard to translate a pattern from written instructions to a Lisp script. However, it would be much easier to convert between visual scripting and Lisp - since there are so few syntactic constructs in Lisp it's possible to convert them easily in both direction.

6.3.1 Language Choice

Rhai[1] is an embedded scripting language implemented as a library for Rust. It's a procedural language with similar syntax to Rust, however its dynamic typing and lack of borrow checking makes it significantly simpler than Rust, and therefore more suited to user scripting. Being a procedural language, it fits better to crochet instructions and is easier to adapt existing patterns to. The problem with this is there is no easy way to convert from a Rhai script into a visual script, as Rhai has far more syntactical constructs than the visual scripting language. However, typically a user will not need to convert from a Rhai script to a visual one, given that they must already understand how the Rhai scripting system works. This means there is only a one-way conversion from a visual script to a Rhai script.

One problem with Rhai is that it is quite verbose, at least compared to written crochet instructions, particularly in the case of repeated instructions. Therefore, Rhai's custom syntax feature was used to create a `rep` statement, that would repeat a given block or function a certain number of times. This might seem like a minor improvement, but in large patterns this helps to make the code much more readable.

```
export let width = 15;
export let height = 15;

for x in 0..width {
    chain();
}
for x in 0..height {
    turn();
    for y in 0..width {
        dc();
    }
}
```

```
export let width = 15;
export let height = 15;

rep width chain();
rep height {
    turn();
    rep width dc();
}
```

Figure 6.3: Comparison of Rhai script without and with custom syntax

6.3.2 Parametrisation

Rhai gives ways to hook into the abstract syntax tree after it is parsed, which is a perfect way to enable variable parametrisation. This allows script variables to be edited in the GUI without requiring knowledge of the scripting system (Req. 2.4.). To facilitate parametrisation the program carries out a post-processing step, which searches for all variables that are 'exported' using `export let` syntax. This allows a script writer to indicate that the variable is a parameter, for example to change the size of a garment. The value given in the script is a default value, used only if the user hasn't modified it in the GUI.

6.4 Pattern Structure

When a script is evaluated, a pattern structure is built up procedurally using functions such as `dc`, `turn` and `chain`. This structure is a graph, where each vertex represents a single stitch and the edges represent the relationships between those stitches, similar to the method laid out in Seitz et al.'s work on crochet charts [4], further described in Section 2.3.

In short, there are two main types of edges between nodes - a 'previous' relationship, and an 'insert' relationship. 'Previous' edges point to the last node to be worked in the current row. Following the chain of 'previous' edges should result in a flat list of every stitch in the piece. 'Insert' edges point to the stitch in the previous row into which the current stitch inserts. That way, a flat crochet piece can be represented as a mesh of nodes, where 'previous' edges connect rows and 'inserts' edges connect columns.

This structure allows for great versatility in pattern representation. Increases can be represented with multiple stitch nodes having an 'inserts' edge into the same stitch. Decreases can be represented with a single stitch node having 'inserts' edges into multiple consecutive nodes.

[1] `https://rhai.rs/`

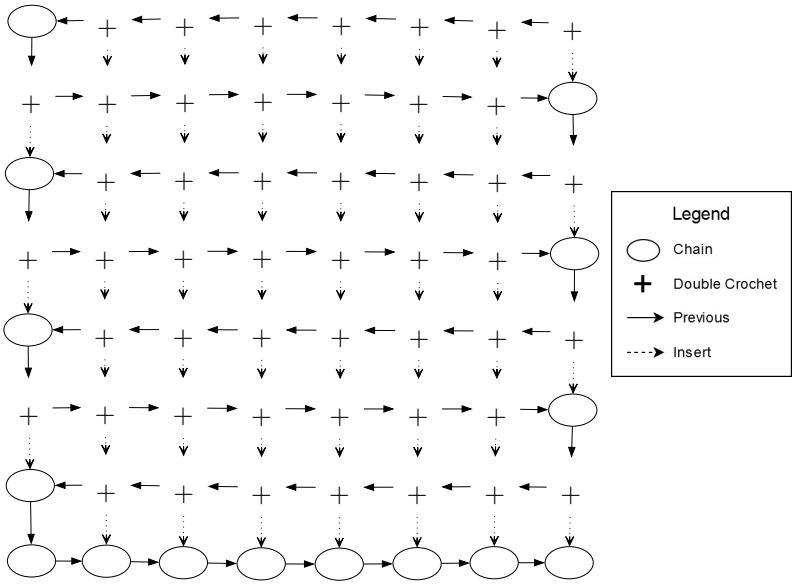


Figure 6.4: The crochet graph of a flat square of double crochet.

This is extended by 'slip stitches', a type of edge that just connects two existing stitches together without creating a new node between them, for example allowing the ends of circular rounds to be joined together. Additionally, the concept of 'holes' in Seitz et al.'s work is present as 'chain space' nodes, which are not stitches in and of themselves, but represent a hole in the work that stitches can insert into. They are positioned using their 'neighbours' edges, which define the stitches that surround them.

However, while Seitz et al.'s work focuses on a system where users manually place and connect stitches, this project allows the automatic generation of a crochet graph from a procedure given by the textual scripting. To do this, there are a number of elementary operations that build up the pattern:

- `chain()` creates a chain stitch, which is represented by a single node and an edge connecting to the previous stitch.
- `new_row()` starts a new row, marking the new insertion point as the first stitch in the previous row.
- `turn()` starts a new row, but marks the new insertion point as the *last* stitch in the previous row. Turning also marks the pattern as being worked in alternating row order[2].
- `skip()` moves the insertion point along to the next node in the last row. If the work is in alternating row order, then it should be the previous node in the last row instead.
- `dc()` creates a double crochet stitch, which is represented by a node and two edges - one connecting to the previous stitch and another connecting to the current insertion point. Then call `skip()` to skip to the next insertion point.
- `dc_noskip()` is the same as `dc()` but doesn't call `skip()` at the end.
- `dec()` creates a double crochet stitch with two insert edges rather than one, then calls `skip()` twice.
- `inc()` simply calls `dc_noskip()` then `dc()`, which has the effect of working two stitches into the same insertion point.
- `set_insert(into)` sets the current insertion point to any arbitrary stitch, allowing one to skip any number of stitches or work into chain spaces.
- `slip_stitch(into)` takes in a reference to another node in the graph and creates a slip stitch edge to it.

[2]This means each row works in the opposite direction from the previous row, as the work is turned 180°

- `start_ch_sp()` and `end_ch_sp()` keeps track of the nodes that are worked in between them and create a 'chain space' node with all those stitches as its neighbours.
- `change_color()` changes the colour of the yarn for all following stitches.

These functions all apply to the `Part` structure, however a pattern can consist of multiple 'parts' all sewn together. This is done using the `sew(a, b)` function which produces 'sew' edges between the pairs of two ordered lists of nodes in separate parts.

6.5 Graph Layout

In order to turn a crochet graph into a 3D model, each node in the graph must be assigned a position in 3D space. This is done using a graph layout algorithm, which optimises these positions to best approximate the surface that the crocheted object would have.

The exact layout is affected by the *gauge* of the crocheted piece, that is to say, the ratio of rows and columns per unit of length. Typically with double crocheted pieces, rows are more dense than columns, that is to say, more rows can fit in a given length than columns.

There are a few requirements for this:

- Stitches in a given row should be roughly the same distance from their neighbours.
- Stitches should be placed a distance equal to the gauge away from stitches they insert into.
- Average curvature along the surface should be minimized as long as the first two conditions are met.

The first two requirements ensure that stitches are placed at the right distances from each other, and the last requirement ensures that flat pieces don't curve unnecessarily, while still allowing intentionally curved pieces to curve. The exact algorithms used to do this are detailed more in Section 7.2.

However, with the algorithm used, the third requirement isn't always perfectly satisfied on every pattern. Luckily, it wasn't too difficult to adapt the graph layout algorithm to only work in two dimensions, so a strict '2D mode' was implemented for pieces that are intended to be flat. This allows complex flat patterns like doilies and 'granny squares' to be modelled without unnecessary curvature.

6.6 Polygon Generation

When choosing how to render a pattern, there were a few options. At first, the pattern was rendered as a wireframe of the graph structure. This involves rendering each edge in the graph as a straight line between its two vertices' positions in the graph layout. This was the simplest method, and was used for the majority of development before being replaced with a more sophisticated method.

One more sophisticated option was to render each cycle of four nodes in the graph with a single quad. This would work well with flat pieces, since they appear as a grid of said four-node cycles, however this lacks flexibility when it comes to more complex shapes. For example, the initial starting chain of a piece is made of a single string of stitches rather than a grid of quads - therefore, this would be skipped in rendering. Additionally, any increases or decreases show as a three-node cycle, which would also be skipped.

Additionally, some patterns might include four-node cycles that aren't supposed to be filled in the same way. For example, the v-stitch pattern (Figure 6.5) involves skipping every other stitch and using chains to make up the difference in length

Another option would be to model the yarn itself as a spline and render that to the screen. This way, each stitch can have a start point and an end point to which the yarn from the previous and next stitches connect, and insert points where the yarn threads through a stitch on the previous row. Then, each stitch type can have a static set of control points associated with them, representing the way the yarn loops around itself to make that stitch, and this can be transformed and composed for each stitch.

However, for this to be effective the number of control points in the total spline would be very high, particularly if the pattern is large. Then, to render the whole spline each segment would need to be

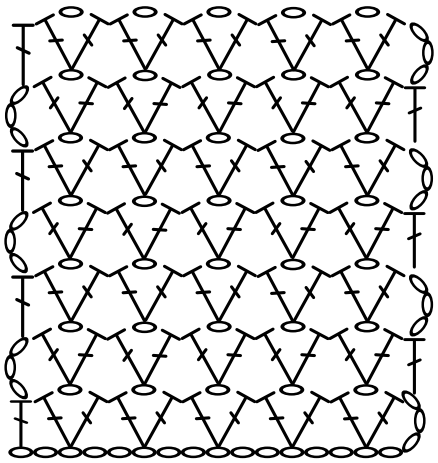


Figure 6.5: The v-stitch pattern, which would be unable to be represented properly by the cycle detection method.

converted into a number of polygons to extrude a cylinder across the path, resulting in a huge number of polygons being rendered on screen at once and can cause unacceptable slowdown for the end user.

Therefore, the approach that was used was much simpler - each stitch would be represented with a single quad, extending from its top edge to the top edge of the stitch it inserts into. Each quad must also face away from the surface it makes, which is done by calculating a tangent vector of the surface and offsetting the corners by that vector. This has its own issues - sometimes quads will overlap, for example for increases where multiple stitches insert into the same place, and sometimes gaps will be left, in decreases for example.

6.7 Interaction Design

Overall, while the app was designed as a proof of concept rather than a user-facing product, it was still important to utilise standard design principles. As such, standard GUI layout features like a menu bar and tab panels were used to make it more intuitive.

The application has four main components:

- The view pane, where the generated model is rendered. This has standard 3D orbit and zoom controls, so a user can see all parts of the model. (Req. 3.3., 3.4.)
- The code view, where the user can write Rhai code to create a pattern.
- The parameters view, where the user can edit exported parameters from the Rhai script.
- The visual view, where the user can visually script their pattern.

The view pane is always visible on the right hand side of the window, while the other three views can be switched between using a tab bar on the left hand side pane. This left pane is resizable so more space can be given to the view pane, or to the code view. Additionally, it's possible to load a Rhai pattern script from a file, using the `.ph` extension (Req. 2.1.), as well as saving those files (Req. 1.6.). A list of example scripts is also included with the application, which were also used for functional testing.

Chapter 7

Implementation

7.1 Crochet Graph Construction

One of the key implementation details is the way the crochet graph is constructed from the list of operations given in a textual script. The graph structure itself was implemented using Rust's `petgraph`[1] crate, which uses an adjacency list representation, and implements construction by adding nodes and edges, as well as some common algorithms like Dijkstra's for pathfinding, which is used later on in the graph layout procedure.

The graph itself is stored in the `Pattern` struct, but most of the crochet operations are in the `Part` struct. A pattern can contain multiple parts, so the `Pattern` struct is intended for root-level operations like part creation and sewing. The graph is stored under a read-write lock (`RwLock` in Rust) so multiple threads can access the same graph, but mutating the graph locks it to all readers or other writers. Then, each `Part` stores a shared reference (`Arc` in Rust, standing for atomic reference counter) to the parent `Pattern`.

The `Part` struct tracks some state in order to construct the graph sequentially. All references to nodes are stored as indices in the graph structure.

- `start` - the first stitch in the part.
- `prev` - the last stitch that was worked.
- `insert` - the stitch that the next stitch should work into.
- `direction` - either 'forward' or 'reverse', indicating whether the piece is in alternating row order.
- `rows` - a vector of rows, where each row is a vector of stitches in the order they were worked.
- `current_ch_sp` - a vector of each stitch being worked during a chain space.
- `ignore_for_row` - tracks whether the next stitch should be included in the current row, when a stitch should be automatically skipped when working the next row.
- `current_color` - the current yarn colour being used for stitches.

Most operations involve creating a stitch node, then adding edges corresponding to its 'previous' and 'insert' relationships, then skipping to the next insertion point. However, each operation also has a `noskip` version, indicated in Rhai by an underscore at the end of the function name (`dc_()` vs. `dc()`), meaning that multiple stitches can be worked in the same insertion point. This is also useful when inserting into chain spaces, since they aren't part of any row and thus a skip operation will fail to find the next insert.

Originally, the skip operation was implemented purely by using the graph structure, taking the current insertion point and finding the incoming edge with a 'previous' relationship. This caused complications in alternating row order, because the next insertion point should actually be the previous stitch, since each row works in reverse order from the last row. Therefore, once the end of the row was reached it could wrap to the row below it, with no way to tell that it was at the row below it. Instead, I just keep track of each row as I build it in the `rows` vector. This also helps with debugging and testing as it was clear where each row started and ended. Additionally, it allows certain stitches to be ignored when the next row intentionally shouldn't work into them. I can also expose to Rhai the `row()` method to return the current row, giving users an easy set of stitches to sew to another part.

[1] `https://docs.rs/petgraph/latest/petgraph/`

7.1.1 Graphviz Output

Since this part was developed before the graph layout or model generation, it was important to find a way of testing that the patterns it was producing were correct.

Since the `petgraph` crate is a generic graph data structure, it also includes convenience methods such as DOT export functionality, the format used for the Graphviz[2] package. Using my own custom graph representation rather than a pre-existing module would mean I'd need to implement this myself, which would be error-prone and time-consuming. Using a more modular approach and building on libraries means that this aspect of development was much easier.

This feature allowed me to export the crochet graph in DOT and visually check that the output is as I expected. Particularly, the 'neato' layout algorithm is similar to my graph layout module, meaning it was particularly useful to visualize the result before my own renderer was in place.

7.2 Graph Layout Algorithms

The next part of the implementation was generating a valid graph layout based on this crochet graph. This involves giving each node a position in 3D space, and trying to optimise these positions so that the piece forms the right shape. As explained in Section 4.2, since the accuracy requirement (Req. 3.1.1.) is subjective, manual testing was performed for this section. During this, the model was displayed as a simple wireframe, where each edge is an edge in the resulting graph. This was a temporary solution, until a more sophisticated rendering process was created.

While there are a lot of graph layout algorithms, most of them are used for laying elements out visually in two dimensions, and don't concern themselves with enforcing distance constraints. It's therefore important to focus on only the algorithms that function in three dimensions and intend to enforce specific distances between nodes.

7.2.1 Force-Directed Graph Layout

The typical distance-based graph drawing algorithm is a force-based one, where each edge is modelled as a 'spring' between nodes, with a resting length. Each spring applies a force to those nodes in the direction of the signed difference between the edge's length and its resting length. Then, for each iteration, each node is moved according to the sum of all forces applied to it. This is fast, since each node's movement only depends on its direct neighbours, but this also means that nodes that aren't neighbours can be placed anywhere relative to one another. This can cause the algorithm to introduce folds, curvature and overlap that aren't desirable.

Fruchterman and Reingold[9] introduced to this process an additional 'repulsion' term, where non-neighbour vertices are repelled from each other with an additional force with magnitude proportional to the inverse square distance. This does increase the runtime, since the sum of forces now includes every other node in the graph, however it's possible to optimise, since beyond a certain distance the repulsion term is negligible. While the majority of folding and overlap is fixed by this addition, the locality of this repulsive term still means that unnecessary curvature can be introduced across the whole surface.

Generally, these force directed approaches are fast to compute, but they require a lot of iterations to converge to a reasonable output. It's easy for them to get stuck in local minima due to a relatively small step size, and thus it can be hard to tell when the algorithm has reached a sensible layout without visual inspection.

7.2.2 Kamada-Kawai Graph Layout

An alternative algorithm was proposed by Kamada and Kawai in 1989[10], where "graph theoretic distance between vertices in a graph is related to the geometric distance between them in the drawing". The algorithm involves calculating the all-pairs-shortest-path (APSP) of the graph, and then trying to optimise the distance between each pair of vertices to match the shortest-path distance between them. Kamada and Kawai do this by optimising a total energy function using a localized 2D Newton-Raphson

[2] `https://graphviz.org/`

method, fixing the position of one node at a time. At each iteration, it moves the vertex whose gradient is highest in the direction of its negative slope, which gradually converges on a sensible layout.

While this algorithm produces a more cogent layout than a simple force-directed algorithm, and is less likely to result in unnecessary curvature, it still takes a long time to converge, and the upfront cost of computing the APSP of the graph is extremely high for larger graphs, typically in $O((V+E)V \log V)$ time if done using Dijkstra's algorithm on each vertex, and $O(V^2)$ space to store the pairwise distances. This is slightly better for this application, since crochet graphs are typically sparse: that is, they have $E = O(V)$, since each vertex only typically has 1-3 outgoing edges, therefore the time complexity is only $O(V^2 \log V)$.

7.2.3 Stochastic Gradient Descent

The solution that was used in the end was based on Zheng et al.'s work on graph drawing using stochastic gradient descent (SGD) [11]. In essence, this approximates the gradient descent approach of the Kamada-Kawai algorithm by repeatedly satisfying constraints on individual pairs of vertices. Rather than attempting to calculate the full gradient for each vertex, the gradient is approximated for each pair by assuming the gradient for every vertex not in that pair is zero.

Satisfying a constraint between two vertices is as simple as moving each vertex towards each other by a vector equal to half the difference between the current distance and the ideal distance.

$$\mathbf{r} = \frac{||\mathbf{X}_i - \mathbf{X}_j|| - d_{ij}}{2} \frac{\mathbf{X}_i - \mathbf{X}_j}{||\mathbf{X}_i - \mathbf{X}_j||} \tag{7.1}$$

However, naively doing this for every pair doesn't lead to convergence, since there's no limit on the distance a vertex should move, and in most graphs it's not possible for every pair of vertices to match the shortest path distance exactly.

Therefore, SGD attempts to satisfy this constraint, but for each full iteration though all pairs, the step size η decreases to zero. This means that the process converges, however it's possible that the final result is not a global minimum. SGD also gives a weighting factor to each pair of vertices, which in the typical solution is equal to the inverse square distance between them, in order to offset the extra priority given to longer paths, since the square of the difference is used in the stress function. However it's also possible to change this weighting factor to prioritize other properties, for example to focus on specific nodes - although, this function is not used in this project, and the weight is set to d_{ij}^{-2}.

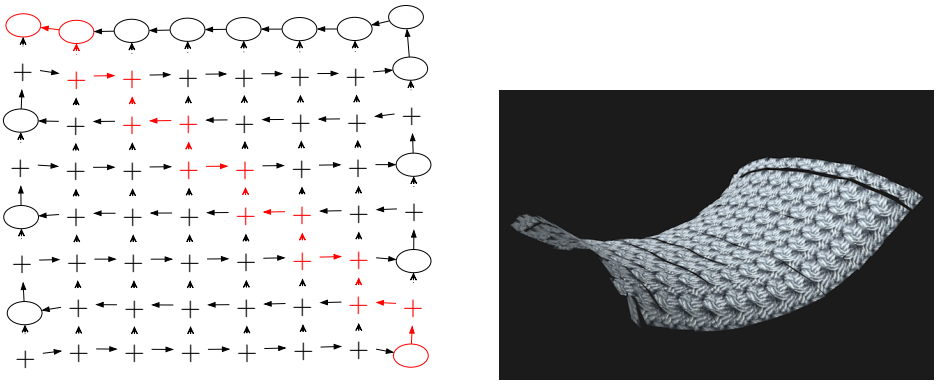
The work by Zheng et al. proposes a modification to this that limits the coefficient of $\mathbf{r}$ to 1, since it shouldn't be possible for a vertex to move further than the constraint allows in any step. This allows the algorithm to use step sizes starting greater than 1, which can lead to faster convergence and makes the algorithm more likely to find a global minimum. They also analyze the step size annealing schedule, that is, the pattern of step sizes to follow to produce the best result. They propose both a schedule for a fixed number of iterations and one for a process that can run indefinitely - here I use the fixed iterations schedule since the process should finish in a fixed time. Experimentally, they determine that the best performing schedule is an exponential decay in the form $\eta_{\max} e^{-\lambda t}$, where λ is a constant that fixes $\eta(t_{\max} - 1) = \eta_{\min}$. This can take longer to converge than some other schedules, but produces lower stress overall.

Finally, it's important that the list of pairwise terms is shuffled after every full iteration, since using the same order every time can get stuck in local minima. The SGD paper also suggests a few ways of minimizing the computational cost of reshuffling the list every time - namely, just shuffling the list twice and alternating between both shuffles. This is the implementation this project uses, since it's significantly faster than shuffling every iteration, and performs almost just as well.

This is about the same in terms of time and space complexity as the Kamada-Kawai algorithm, since it still requires the upfront APSP calculation. In the actual optimisation step, Kamada-Kawai takes $O(V)$ time to compute a given gradient, whereas SGD takes $O(V^2)$ time to iterate through all pairs. However, Kamada-Kawai typically takes $\Omega(V)$ iterations, since all vertices need to be optimised at least once, whereas SGD can use a constant number of iterations and still converge upon a decent result.

7.2.4 Triangulation

One problem that was found with the SGD algorithm was how it deals with diagonal paths. A flat crocheted sheet is represented as a two-dimensional lattice in the graph structure. As such, the shortest path distance between two points is equal to their Manhattan distance in this lattice. However, SGD intends to optimise the layout such that the Euclidean distance is equal to this distance. This means that while straight paths on the lattice are optimised correctly, diagonal paths from corner to corner have their distances overestimated, resulting in a 'tugging' effect at the corners. This introduces a saddle shape to a flat piece, which is not intended.

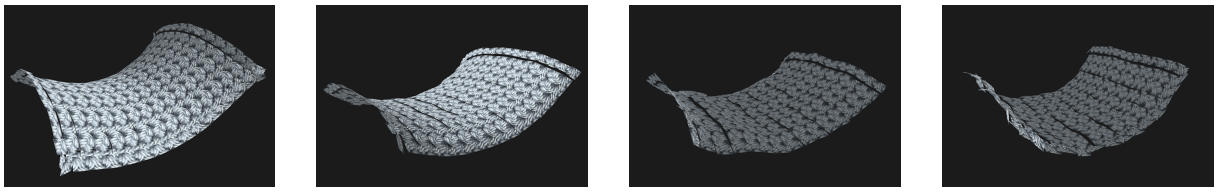


(a) The graph of a flat crochet piece, with a diagonal path highlighted.

(b) The resulting model, with a distinctive saddle shape.

Figure 7.1: A demonstration of the 'saddle' effect on flat crochet pieces.

One potential solution to this is the weighting factor used by SGD, which intentionally de-emphasises longer paths. Increasing the exponent of this factor would further restrict long paths from having an effect on the result, mitigating this effect. However, this has the effect of de-emphasising straight paths along the grid as well, meaning curvature can show up in unintended places.



(a) $w_{ij} = 1$ (b) $w_{ij} = d_{ij}^{-2}$ (c) $w_{ij} = d_{ij}^{-4}$ (d) $w_{ij} = d_{ij}^{-6}$

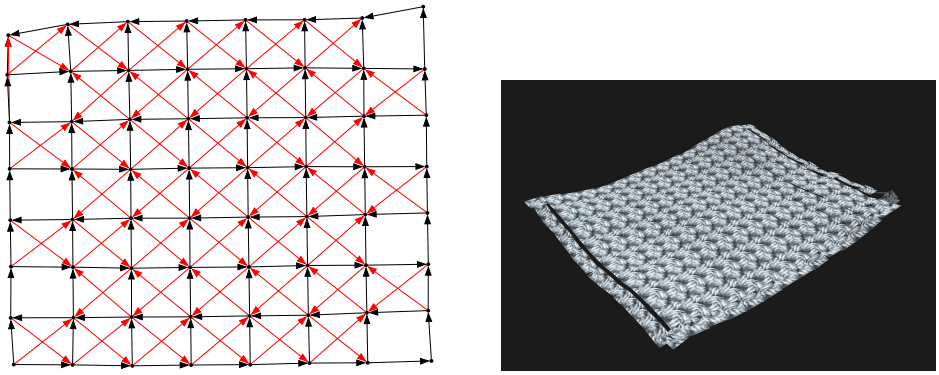
Figure 7.2: The effect of varying the weighting exponent for long paths. The saddle effect is less pronounced, but pieces curve inwards instead, and the layout is less stable.

In order to solve this, the application adds 'shortcuts' between these diagonal nodes, extra edges which allow the pathfinding algorithm to match the Euclidean distance more accurately. Notably, some stitches are turns, which are represented as chains. Since chains don't insert into any stitch, it's not possible to give them diagonals using this method.

This does remove the saddle effect as shown above, but has adverse effects on intentionally curved pieces. Since the diagonal shortcuts have an enforced length, this removes flexibility from stitches. Particularly, increases and decreases have a naturally trapezoidal shape since the top row must necessarily be longer than the bottom row. This can cause some difficulties in causing curved shapes to converge. As such, a rudimentary force-directed layout step is applied with a small step size, after the main SGD step, to try to correct this inflexibility.

7.2.5 Normalisation

As a final post-processing step, the algorithm attempts to normalise the position and orientation of the piece. Since SGD starts with a random seed it can produce a different result every time, and this can make it confusing to view, since there is no canonical orientation.



(a) A lattice of stitches, with added shortcuts. (b) The resulting model, with only minor curvature.

Figure 7.3

This normalisation step simply translates the whole model from its center of mass, that is, the average position across all vertices, to the origin, and rotates the model so the normal vector of the closest vertex to the center of mass is facing upwards. This is rudimentary, and doesn't perform particularly well for non-flat pieces - a more complex process such as a minimal bounding box algorithm using rotating calipers was considered, but was determined to be out of scope.

7.3 Rendering

Rendering the model was the next step in the process. The render pipeline was set up in WGPU, since `egui`, the GUI library the project uses, has a way of setting up arbitrary graphics using WGPU. There are a few different components to implement here:

- A model, represented as a matrix of vertex data and a matrix of indices representing the triangles in the model.
- A transformation matrix, made up of component model, view and projection matrices.
- A shader, which describes the calculations to be made on the GPU, and its corresponding binding groups, which describe the data to be supplied to the shader.

This is all that was needed for a rudimentary renderer, although later I also implemented texturing, for the model to look a little more realistic.

During testing of the graph layout algorithm, a very basic wireframe renderer was used. Rather than taking in a set of triangles, it simply took in a list of edges to render as straight lines. However, later on the more sophisticated method described in Section 6.6 was implemented instead. For each node in the graph, a rectangle is created connecting it to each of its outgoing 'insert' edges, or in the case of a chain stitch which has no insert edge, its outgoing 'previous' edge. A tangent for the rectangle is calculated by averaging the other outgoing and incoming edges, which also determines the width of the rectangle. This process is approximate, but it works well enough for most nodes in the graph.

To texture the models, an image was designed that would loop seamlessly with itself to form a grid, by taking a photo of a double crochet square up close and cropping out two stitches - since a square is worked in alternating row order, it means that the front and back side of a stitch can be captured in one image. Then, some image manipulation was done to make sure it looped seamlessly, and then was imported into a texture buffer. Each rectangle in the model has both a front and a back side, and so the texture coordinates or UVs are set to match either the top half or the bottom half of the image, depending on the side. This has the effect of making the model relatively seamless, although some small gaps can appear due to inaccuracies in the rectangle generation.

7.4 Visual Script Representation

The visual scripting system required the development of an abstract syntax tree - a recursive tree of expressions that represent the structure of the script. While this would be relatively simple in most languages, because Rust has no garbage collection and is very strict about memory management, it can be hard to develop recursive data structures. A typical struct in Rust must sized - it must have its size known at compile time. Given that a recursive structure can have theoretically infinite size, the nested part must by under some indirection, like a pointer. Then, when creating the tree, each node can be allocated on the heap, and deallocated once the pointer is dropped - this is implemented by the `Box` type in Rust, similar to `std::unique_ptr` in C++.

The problem with this, however, is that it requires each node to be allocated separately, potentially in totally different sections of memory, meaning that large syntax trees are more likely to break cache locality, and also require a large amount of pointer indirection. Therefore, Adrian Sampson suggests in [12] that it's better to store an AST in a flattened manner, where all nodes are stored in a vector, and pointers to child nodes are instead stored as indices into this vector. This means that all nodes are stored contiguously, retaining cache locality, and a full traversal of the tree can be done by simply looping through the vector, assuming order is not important.

7.5 Multithreading

Given that the graph layout algorithm can take a long time for large patterns, it's important to make sure the user can still interact with the program while the model is being generated (Req. 5.4.). Running the algorithm on the main thread would cause the program to freeze as the UI rendering would be paused. Therefore, this work is performed on a worker thread instead, spawned with a copy of the Rhai pattern code as a parameter. The script evaluation, graph layout, and model generation is all performed on this thread, and the main thread polls whether the worker is finished each frame. Once it is, it can send the vertex data to the main thread, which moves this data into the vertex buffer to be rendered. This way, the application remains interactive while the algorithm runs in the background. Rust's guarantees about ownership and mutable exclusivity also ensures that no data races are possible, so the process is entirely safe.

Chapter 8

Evaluation

8.1 Evaluation Against Requirements

To evaluate the final product, I compare it with the requirements set out in section 3.3.

> 1. **Creation** – A *pattern creator* should be able to:
>   1.1. write program code representing a crochet pattern, with standard crochet instructions (double crochet, chain, increase, decrease, etc.), repeated shorthand for multiple instructions, and subroutines to define custom stitches
>   1.2. define parameters for the end user to modify
>   1.3. sew multiple distince crocheted parts together
>   1.4. implement each of these tasks in a simple and readable manner

All of the basic feature requirements are met here: standard stitches (`dc()`, `chain()`, etc.), repetition (`rep 15 { chain() }`), subroutines (using standard Rhai functions), parameter definitions (`export let`), and part sewing (`sew`).

Requirement 1.4. asks whether all these tasks are simple to implement and easy to read. While this is a subjective criterion, most of these features are implemented with short statements or function calls, resulting in relatively readable scripts even when compared with their written instructions (Fig. 8.1). That said, reading the scripts does typically require technical knowledge of the scripting system, so they aren't suitable for direct distribution to a crocheter.

```
magic_ring();
into(mark());
chain();
rep 6 dc_();
                                row 1: ch, dc 6 into magic ring.
new_row();                      row 2: inc 6.
rep 6 inc();                    row 3: (dc, inc) x 6.
                                row 4: (dc 2, inc) x 6.
for j in 1..=5 {                row 5: (dc 3, inc) x 6.
    new_row();                  row 6: (dc 4, inc) x 6.
    rep 6 {                     row 7: (dc 5, inc) x 6.
        rep j dc();
        inc();
    };
}
```

Figure 8.1: Comparison of Rhai script and written crochet instructions

The main exception to this simplicity is the `sew` functionality. Since the sewing system requires a paired list of the exact stitches to match up to one another, it requires precise selection of the exact stitches to sew to a piece. If two pieces are being sewn together along a row or round line, this is easier because the `row()` function is exposed, which returns a list of all stitches in a row. However, when a part needs to be sewn onto the side of a piece, that is to say, across multiple rows, then the stitches need to be selected manually. The typical pattern for this is storing all relevant rows' stitches in a two-dimensional list and picking the indices from this list to sew to the other part (Fig. 8.2). This mostly involves trial and error, and results in a messy looking pattern. If further work were to be done, it could be useful to automatically find valid sew points algorithmically, possibly using a breadth-first search to find all stitches within a certain distance from a given stitch, and selecting these in a sensible order.

Overall, while the scripting system is relatively intuitive for single-part patterns, as soon as sewing is involved the process relies on too much trial and error to be particularly usable.

```
let rs = [];
rep 5 {
    new_row();
    rep 60 dc();
    rs.push(row());
};

let head_sew = [
    rs[0][29], rs[0][30], rs[0][31],
    rs[1][31], rs[2][31], rs[3][31],
    rs[4][31], rs[4][30], rs[4][29],
    rs[3][29], rs[2][29], rs[1][29],
];

let arm_1_sew = [
    rs[0][19], rs[0][20], rs[0][21], rs[0][22], rs[0][23],
    rs[1][23], rs[2][23], rs[3][23],
    rs[4][22], rs[4][21], rs[4][20], rs[4][19],
    rs[3][19], rs[2][19], rs[1][19],
];

let arm_2_sew = [
    rs[0][39], rs[0][40], rs[0][41], rs[0][42], rs[0][43],
    rs[1][43], rs[2][43], rs[3][43],
    rs[4][42], rs[4][41], rs[4][40], rs[4][39],
    rs[3][39], rs[2][39], rs[1][39],
];
arm_2_sew.reverse();

let foot_1_sew = [
    rs[2][5], rs[2][6], rs[2][7],
    rs[3][7], rs[3][6], rs[3][5],
];
let foot_2_sew = [
    rs[2][-5], rs[2][-6], rs[2][-7],
    rs[3][-7], rs[3][-6], rs[3][-5],
];
```

Figure 8.2: An example of using 2D row indexing to find valid sew points for parts.

1.5. use a visual system to create a crochet pattern, with standard crochet instructions, as above, and repetition blocks for multiple instructions. Visual patterns should be faster to create, and can therefore be simpler than textual scripts

The basic functionality here is present in the Visual View panel - standard crochet instructions, repetition and parameter definitions are all implemented. While visual patterns are particularly simple to create thanks to the reduced complexity, the final feature set is too restrictive to make many different kinds of patterns. Since most procedural constructs like subroutines, state modification and arithmetic aren't present, a visual script resembles a written crochet pattern more so than a textual script. However, some basic crochet operations like changing the insertion point and slip stitches aren't present.

1.6. save and distribute the pattern as a file to other users

2. **Usage** – A *pattern user* should be able to:
   2.1. open a pattern file created by a *pattern creator*
   2.2. modify the parameters defined by the pattern creator without editing the pattern itself
   2.3. export the list of instructions to craft the real article
   2.4. perform all these tasks with no technical or programming knowledge

For a pattern user, the software is relatively simple to use. A creator can save the pattern as a '.ph' file, and upload that file online for others to use. Following this, a user can open the file in Polyhook either by passing the path as a command-line argument or using the 'Open' option in the 'File' menu. Then, they can modify the parameters exported by the creator using the 'Parameters' tab, without needing to understand the `export let` syntax used in the script. The parameters are currently limited to only integer values, but the system could be extended to allow for editing of string, decimal and even colour values, although some work may need to be done to ensure the type is correct, since Rhai has a dynamic

type system and thus would infer the type from the default value.

The main feature that is not present here, though, is Req. 2.3., which was unable to be implemented at the time. This feature is more complicated than it seems, mostly because the implementation includes no intermediary step between a script and its representation as a crochet graph. To generate instructions, either the script would have to compile to an IR which can then evaluate to a graph or a set of written instructions, or there would have to be a way of converting a crochet graph into instructions.

The first approach is difficult because the script is tightly coupled with the crochet graph system. For example:

```
// ...
let previous_row = row();

new_row();
for stitch in previous_row {
    dc();
}
// ...
```

This script retrieves part of the current graph state (the current row) and performs a certain number of instructions based on that state. In an IR, this can't be compiled to a flat list of crochet operations, since the number of instructions depends on state that is only available once the script is evaluated to a graph. This either means that all the graph evaluation state must also be present during the conversion to a written format, or that statements the depend on state need to be representable in this format as well (e.g. "dc for each stitch in previous row", or "dc across", which is a common crochet instruction). The problem with this is that this instruction can be implemented in many different ways in Rhai:

```
for stitch in previous_row { dc(); }
// or
for _ in 0..previous_row.len() { dc(); }
// or
rep previous_row.len() { dc(); }
// or
let i = 0;
while i < previous_row.len() {
    dc();
    i += 1;
}
```

All of these (and many more) would have to be converted to the same written instruction. Since there is no single canonical form for this instruction, and no way to reduce this Rhai code to a simpler form, it's extremely hard to write an algorithm that detects this form and converts it to the correct written format.

The other option is generating written instructions directly from a crochet graph. While this approach may seem more promising, this suffers from a similar problem in that a given crochet graph result can be generated from many different patterns. As referenced in Section 2.3, Edelstein et al.[8] posit that a final crochet work is analogous to an execution trace of a pattern. While it may be possible to reconstruct the pattern from the trace, this would likely rely on heuristics and approximations rather than a concrete algorithm. Due to the difficulty of solving this problem, after some time it was decided it would be out of scope for this proof-of-concept.

3. **Visualization** – Any user should be able to:
   3.1. visualize the currently opened pattern as a three-dimensional model, which should:
      3.1.1. resemble the real article created by following the instructions in the pattern
      3.1.2. not have any unnecessary curvature or folding, even if the real article can fold or bend
      3.1.3. if three-dimensional and enclosed, inflate as though stuffed with material

This feature was implemented using the right-hand panel in the window, which renders a three-dimensional model generated from the crochet graph representing the scripted pattern.

In terms of accuracy, simpler patterns are typically better. Table 8.1 shows comparisons between the real articles and generated models in Polyhook. Flat, square patterns in particular are the most accurate, with very little unnecessary curvature or folding. However, accuracy can drop when crocheting in rounds, and often a lot of unnecessary curvature is introduced. Flat circles are particularly inaccurate, however the strict 2D mode helps to mitigate this. (Fig. 8.3)

When working in rounds, enclosed shapes seen to perform best, although they tend to flatten slightly more than expected at either end. More complex patterns with sewn parts typically have some resemblance to the real article, but sew points tend to 'stretch' out. For example, in row 3 of Table 8.1, the model stretches out at the neck, which should typically be placed much closer to the body. This could be solved when adapting the pattern by adding more sewing points, however as explained before, the sewing process is extremely trial-and-error based, especially considering most written patterns don't provide explicit instructions as to where to sew each part, only to sew them where they look correct to the provided reference image.

> 3.2. restrict the model to two-dimensions, in the case of a flat piece like a doily or shawl

As mentioned above, the strict 2D mode is implemented by restricting all vectors in the graph layout process to only two components rather than three. This helps to constrain patterns that are intended to be flat to two-dimensions, helping with certain kinds of patterns that would be very unstable otherwise.

In particular, this significantly helps with round patterns and certain patterns that don't support the 'shortcut' triangulation explained in Section 7.2.4, namely the v-stitch pattern shown in Fig. 6.5. Since this pattern doesn't use a grid layout it's not possible to add shortcut edges, resulting in an extreme saddle effect as shown in Fig. 8.4. This is mitigated in the 2D version of the pattern, since there isn't a third dimension for the corners to be pulled into. It is possible to see, however, that even in the 2D layout the corners are still slightly overextended.

> 3.3. move the view around the model using the mouse to show all angles
> 3.4. zoom in to the model using the scroll wheel

The orbit and zoom controls are implemented in the final product, using egui's mouse input system and a view matrix to define the viewing angle. One problem with this, however, is that there is a limited space of rotations accessible, since the orbit controls are based on azimuthal and polar angles. While these are sufficient for most viewing angles, the lack of a third degree of freedom - namely, a *roll* component - means that it can be hard to view a model standing upright, especially if the normalization process in Section 7.2.5 results in a model in a poor orientation. A control scheme that involves three components, however, would typically either require three controls to use rather than just the X and Y components of the mouse position, or would rely on holonomy to rotate in the third component, which can be difficult to predict and unintuitive for a user.

> 4. **Flexibility** – The pattern creation system should:
> 4.1. allow for most patterns to be represented using the standard operations
> 4.2. allow fine control over the pattern, if standard operations don't suit their needs

Many different kinds of patterns can be represented using the standard operations: `chain`, `dc`, `new_row` and `turn`. However, for those patterns that can't be made with only these operations, a set of more fine-grained functions is also available, including `mark` (gets the last stitch), `curr` (gets the current insertion point), `into` (sets the current insertion point), and `chain_space` (creates a chain space node with the following stitches).

The main feature this application is missing is the ability add new core stitches. It's currently impossible to work common crochet stitches like trebles, double trebles, and front or back loop stitches. Theoretically, it would be simple enough to implement trebles and longer stitches simply by changing the `gauge`, that is, the number of rows that can be worked in a given length. Front and back loop stitches are more difficult, however, as they often affect the shaping of a piece by adding sharper edges, which is currently not possible with the graph layout approach used. Additionally, implementing other stitches would require a user to be able to supply their own texture for the stitch to look as it would in a real article.

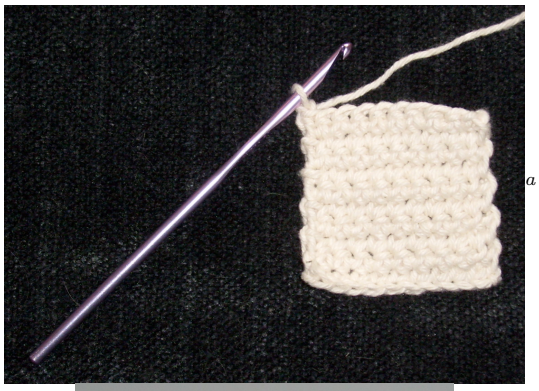
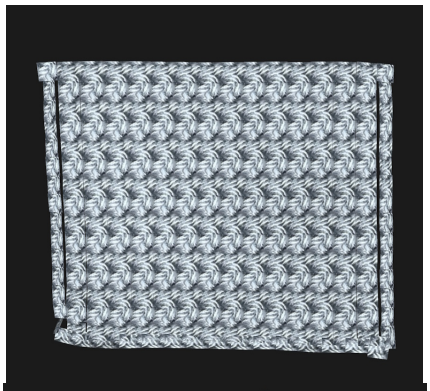
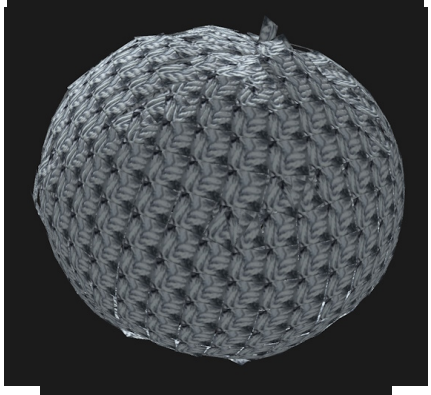
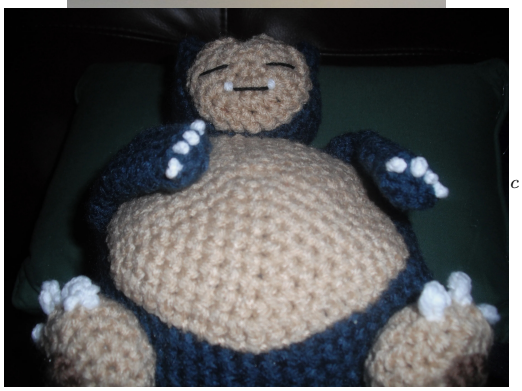
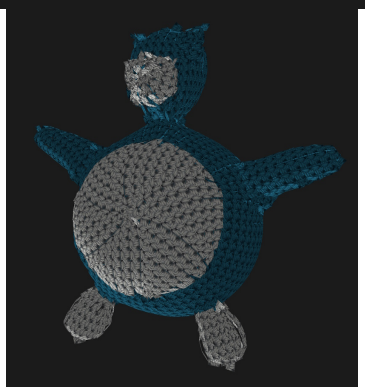
#	Real article	Polyhook model
1	 [a]	
2	 [b]	
3	 [c]	

Table 8.1: Table of comparisons between real articles and the adapted model in Polyhook.

[a] Image from Wikimedia Commons: `https://commons.wikimedia.org/wiki/File:Singleturning.jpg` licensed under Creative Commons 4.0 BY-SA `https://creativecommons.org/licenses/by-sa/4.0/`

[b] Original pattern and image by Ms Premise-Conclusion: `https://mspremiseconclusion.wordpress.com/ideal-crochet-sphere/`. Image licensed under Creative Commons 2.0 BY-NC-ND `https://creativecommons.org/licenses/by-nc-nd/2.0/`

[c] Original pattern and image by Shelly Hedko: `https://epic-yarns.com/2012/01/23/snorlax/` licensed under Creative Commons 3.0 BY-NC-ND `https://creativecommons.org/licenses/by-nc-nd/3.0/`

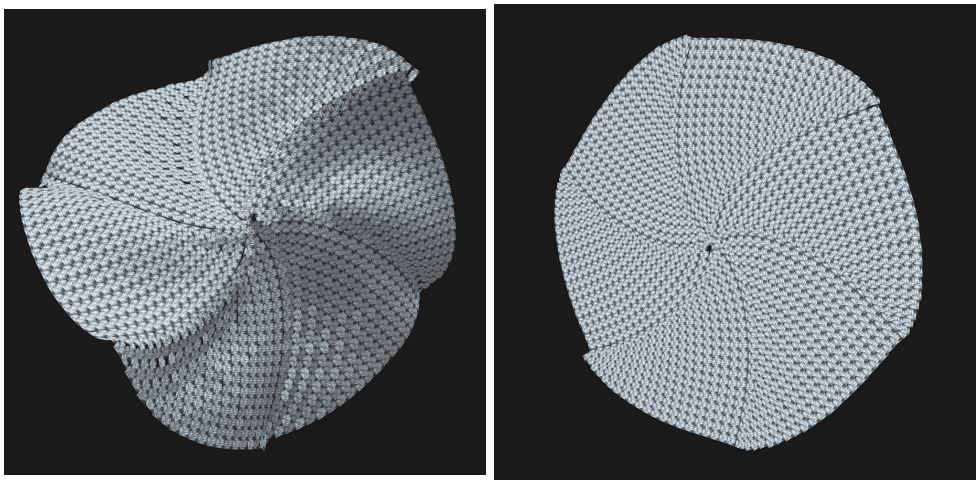


Figure 8.3: Comparison of a ‘continuous rounds’ pattern between the 3D and 2D modes of Polyhook.

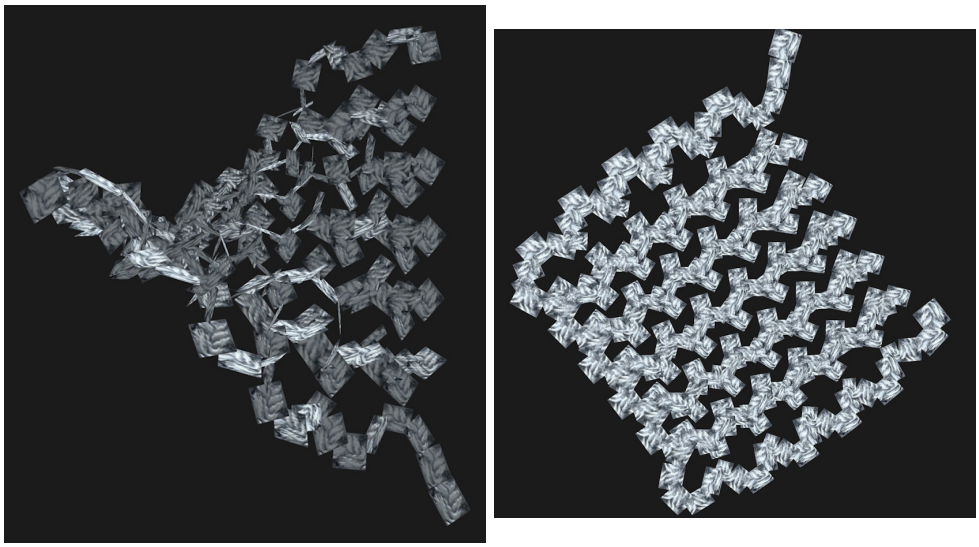


Figure 8.4: Comparison of a ‘v-stitch’ pattern between the 3D and 2D modes of Polyhook.

5. **Usability** – The application should:
   5.1. follow standard UI design practices where possible
   5.2. provide sensible error messages to the user

The UI for this application is designed in accordance with typical design principles (Section 6.7), particularly the parts intended for non-technical users. However, the visual view proved a difficult challenge to make intuitive.

The current implementation allows a user to add steps before and after other steps in sequence using a '(+)' button that appears when the step is hovered over. However, this proved complex since the button would typically obstruct other instructions the user might want to change or modify. Therefore, the button instead moves other instructions out of the way, but this means that often instructions will move out of the way when the user wants to click on them, because another instruction has been hovered over. Adding a short timeout to this behaviour, so the button would remain for a while before disappearing, does help mitigate this, but overall the process is still relatively difficult to manage.

   5.3. take no more than 30 seconds to render a pattern with at most 2500 nodes
   5.4. run at a consistent frame rate and never freeze

Rust has multiple different compilation modes, and under Debug mode the performance is typically significantly worse than under Release mode, since the compiler leaves the resulting machine code mostly unoptimised. I ensured that the performance constraints were at the very least maintained for the Debug build using unit testing, meaning that the performance of the application under Release mode should be even faster than the requirement implies. Figure 8.5 shows how the execution time increases against the number of nodes in the graph, and shows that even in Debug mode, the execution time stays below thirty seconds for a pattern with 2401 nodes.

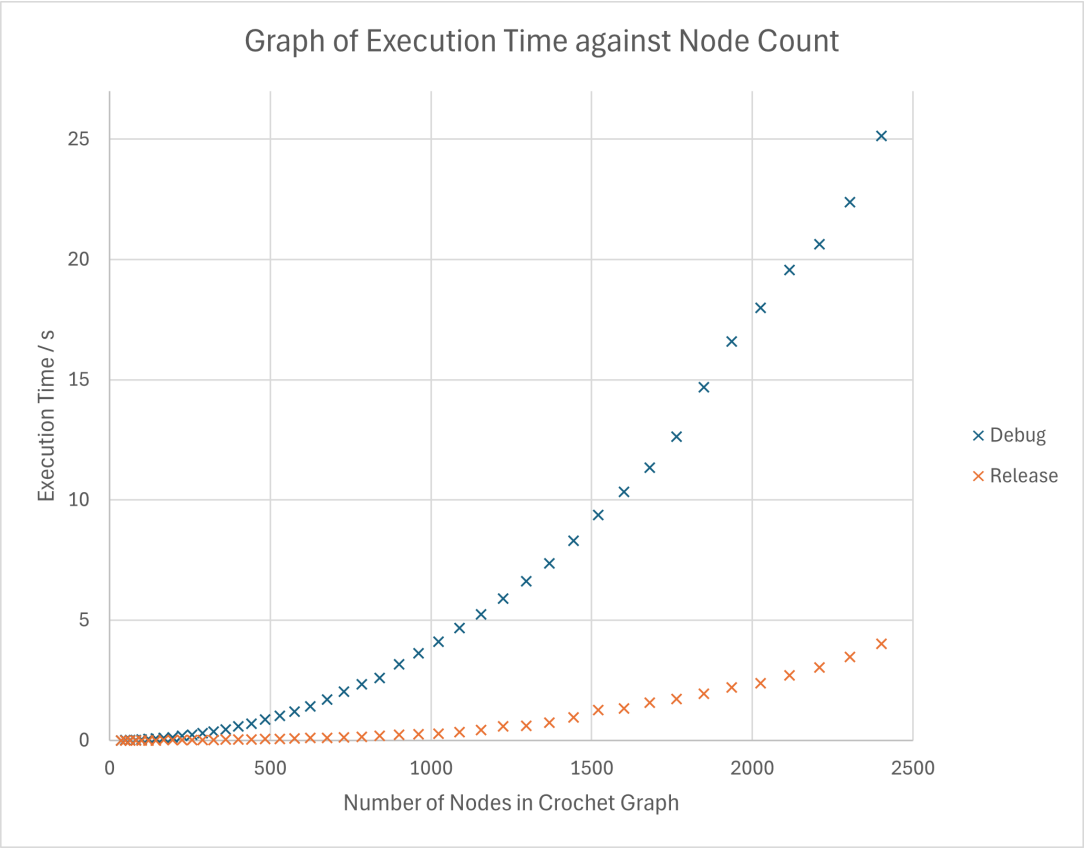


Figure 8.5: Graph showing how the runtime of the model generation process increases with the number of nodes.

Additionally, the multithreading system implemented in Section 7.5 allows the program to keep running and rendering even when generating a model, meaning that the previously generated model can still be

interacted with. Therefore, the frame rate is always relatively consistent and the application will not freeze.

8.2 Evaluation Against Objectives

After evaluating against the requirements specification, I also evaluate against the initial objectives set out in the submitted DOER document. (Section 3.1)

For the primary objectives, the final product includes a standardised data format that can be distributed between users, with support for most standard crochet stitches. However, the ability for users to define their own stitches and extend the format is not present beyond being able to define subroutines to sequence built-in stitches. The final software does allow users to develop patterns parametrically, with repetition and subroutines. While the final product doesn't allow wireframe rendering (since this was built upon to render the model in a more sophisticated manner), it does simulate the expected curvature and shaping of the work.

For the secondary objectives, the lit and textured rendering is present, and the ability to compose parts together to form more complex modes was implemented. However, rendering a pattern to another format was not implemented, although generation of crochet charts could be done by exporting the model to the Graphviz DOT format and rendering the resulting graph.

Unfortunately, none of the tertiary objectives were able to be implemented in the end product. In particular, the instructional mode was a significantly more difficult problem than at first imagined, due to the reasons stated earlier.

Chapter 9

Conclusion

In this dissertation, I design a solution for creating crochet patterns using parametric computer-aided design principles. This was implemented as a desktop application using the Rust programming language, including a textual scripting system using the Rhai scripting language and a visual scripting system.

9.1 Further Work

The system could be further developed in a few key ways:

- Custom stitch support - the ability to define custom stitches with their own dimensions and texturing, allowing pattern creators more flexibility without too much added complexity.
- Instruction export - the ability to export a parametric pattern as a set of written instructions for an end-user to follow.
- Crochet chart export - the ability to export a pattern as a crochet chart, using a similar graph layout algorithm but with symbols in 2D rather than polygons in 3D.
- Visual editor improvements - a more sophisticated visual editor with more features, allowing for flexibility without needing to learn the scripting language.
- Rendering improvements - a better rendering engine might improve the visual style

9.2 Final Evaluation

Overall, the application seems to be accurate and flexible enough for particularly simple patterns. The use-case of Amigurumi is particularly well-served here, since the application works best with simple, enclosed shapes, which includes most of the parts used for Amigurumi patterns. Other use-cases like lacework or textured crochet tend to have much more complicated, intricate designs, which require greater accuracy and greater flexibility than the application currently allows.

In summary, the final product can help to reduce iteration cost by allowing a user to design parts of a crocheted model before starting manufacturing. While the process might not be 100% accurate, even just a basic idea of the shaping of the model resulting from a pattern can help to verify whether a pattern is written correctly or not. A writer can also use Polyhook to verify multiple different sizes of pattern without needing to test each one manually.

Bibliography

[1] A. P. Library, *Crochet stitches*. [Online]. Available: https://www.antiquepatternlibrary.org/crochetstitches.htm.

[2] M. Lambert, *My Crochet Sampler*. D.M. Peyser, 58 John-St., and 363 Broadway. 1847, 1847. [Online]. Available: https://www.antiquepatternlibrary.org/html/warm/E-WM115-003.htm.

[3] M. Pullan, *The lady's manual of fancy work : a complete instructor in every variety of ornamental needle-work*. New York : Dick & Fitzgerald, 1859. [Online]. Available: https://archive.org/details/ladysmanualoffan1859pull.

[4] K. Seitz, J. Lincke, P. Rein, and R. Hirschfeld, *Language and tool support for 3D crochet patterns*. Jan. 2021. DOI: 10.25932/publishup-49253.

[5] J. F. N. Grimmelsmann A. Ehrmann, "Crochet machine," WO002018114025A1, 2017. [Online]. Available: https://depatisnet.dpma.de/DepatisNet/depatisnet?action=bibdat&docid=WO002018114025A1.

[6] G. Perry, "Croche-matic–building a robot for crocheting 3d spherical form," Ph.D. dissertation, 2022.

[7] A. Mathur, M. Pirron, and D. Zufferey, "Interactive programming for parametric cad," *Computer Graphics Forum*, vol. 39, no. 6, pp. 408–425, 2020. DOI: https://doi.org/10.1111/cgf.14046. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14046. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.14046.

[8] M. Edelstein, H. Peleg, S. Itzhaky, and M. Ben-Chen, "Amigo: Computational design of amigurumi crochet patterns," in *Proceedings of the 7th Annual ACM Symposium on Computational Fabrication*, ser. SCF '22, Seattle, WA, USA: Association for Computing Machinery, 2022, ISBN: 9781450398725. DOI: 10.1145/3559400.3562005. [Online]. Available: https://doi.org/10.1145/3559400.3562005.

[9] T. M. J. Fruchterman and E. M. Reingold, "Graph drawing by force-directed placement," *Software: Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, 1991. DOI: https://doi.org/10.1002/spe.4380211102. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.4380211102. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.4380211102.

[10] T. Kamada and S. Kawai, "An algorithm for drawing general undirected graphs," *Information Processing Letters*, vol. 31, no. 1, pp. 7–15, 1989, ISSN: 0020-0190. DOI: https://doi.org/10.1016/0020-0190(89)90102-6. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0020019089901026.

[11] J. X. Zheng, S. Pawar, and D. F. M. Goodman, *Graph drawing by stochastic gradient descent*, 2018. arXiv: 1710.04626 [cs.CG]. [Online]. Available: https://arxiv.org/abs/1710.04626.

[12] A. Sampson, *Flattening asts (and other compiler data structures)*, 2023. [Online]. Available: https://www.cs.cornell.edu/~asampson/blog/flattening.html (visited on 2025-01-06).

Appendix A

Ethics Self-Assessment

UNIVERSITY OF ST ANDREWS
TEACHING AND RESEARCH ETHICS COMMITTEE (UTREC)
SCHOOL OF COMPUTER SCIENCE
PRELIMINARY ETHICS SELF-ASSESSMENT FORM

This Preliminary Ethics Self-Assessment Form is to be conducted by the researcher, and completed in conjunction with the Guidelines for Ethical Research Practice. All staff and students of the School of Computer Science must complete it prior to commencing research.

This Form will act as a formal record of your ethical considerations.
Tick one box

- ☐ **Staff Project**
- ☐ **Postgraduate Project**
- ☒ **Undergraduate Project**

Title of project

Parametric Computer Aided Design of Crochet Patterns

Name of researcher(s)

William Gooch

Name of supervisor (for student research)

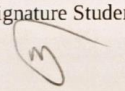
Jason Jacques

OVERALL ASSESSMENT (to be signed after questions, overleaf, have been completed)

Self audit has been conducted **YES** ☒ **NO** ☐

There are no ethical issues raised by this project

Signature Student or Researcher



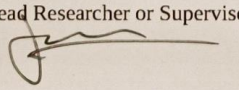
Print Name

WILLIAM GOOCH

Date

25/09/2024

Signature Lead Researcher or Supervisor



Print Name

JASON T. JACQUES

Date 25TH SEPTEMBER 2024

This form must be date stamped and held in the files of the Lead Researcher or Supervisor. If fieldwork is required, a copy must also be lodged with appropriate Risk Assessment forms.

Appendix B

User Manual

README.md 2025-01-15

Polyhook - Parametric Crochet CAD software

Polyhook is a parametric CAD software package for creating crochet patterns. It allows the user to program patterns using the Rhai scripting language and visualize them as a 3D model.

Building

This project is built in **Rust**, and requires the latest nightly version to be installed.

Windows

Download `rustup-init.exe` at https://rustup.rs/, then run and select `2) Customize Installation` and choose default host triple, `nightly` toolchain, and `minimal` profile.

Linux

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -- --default-
toolchain nightly --profile minimal
```

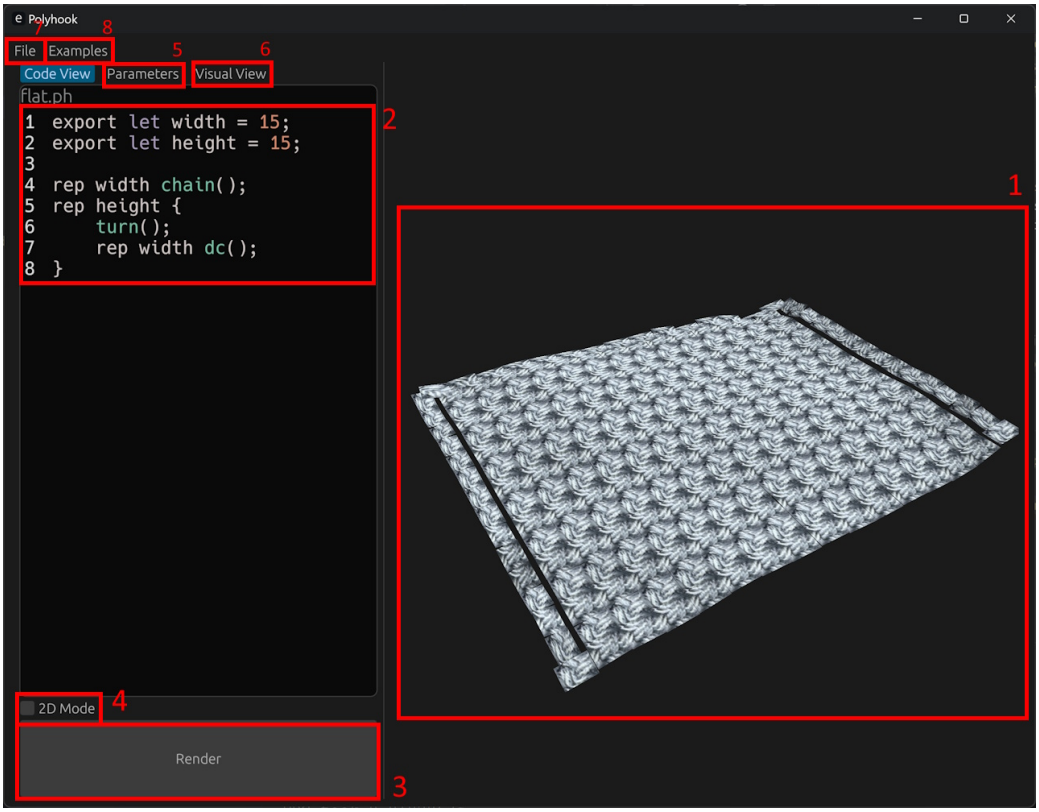
Then, clone the repository and run `cargo build --release` which will build the project.

The executable will be available at `target/release/polyhook(.exe)`

Usage

Start the application with `polyhook`, or with a file path as an argument to start with a file open, e.g. `polyhook hooklib/examples/sphere.ph`.

1	The final 3D model of the written pattern. Click and drag to rotate, or scroll to zoom in.
2	The code view, where a pattern script can be written.
3	Renders the script as a 3D model.
4	Restricts the model to two-dimensions, i.e. a flat sheet.
5	Changes the parameters exported from the written script.
6	Allows you to create a model through visual scripting rather than code.
7	Open or save a pattern as a `.ph` file.
8	Open example patterns included with the software.



Scripting

This software uses the Rhai scripting language to write patterns. For basic syntax, refer to Rhai's documentation.

An overview of each Polyhook-specific function is as follows:

`chain`	Work a new chain stitch
`skip`	Skip the current stitch in the base row
`dc`	Work a double crochet into the current insertion point, then skip
`dc_`	As above, but don't skip
`new_row`	Start a new row
`turn`	Start a new row and turn, working in alternating row order
`turn_`	As above, but don't skip the first stitch of the row
`dec`	Work a decrease
`magic_ring`	Start the part with a magic ring
`ss`	Work a slip stitch into the given stitch

`into`	Move the insertion point into the next stitch
`mark`	Mark the last stitch worked and return a reference to it
`curr`	Return a reference to the current insertion point
`row`	Return a list of references to all stitches in the current row
`chain_space`	Create a chain space with all stitches worked in the given function or closure, and return a reference to it
`ignore`	Work all stitches in the given function or closure, without adding them to the current row
`new_part`	Create a new part, disconnected from the last one
`sew`	Take two lists of stitches, and sew them together pairwise

Appendix C

Testing Summary

Test Name	Status	Runtime (s)
`hooklib::parametric::tests::test_example_flat`	Pass	0.0000623
Tests that a parametric pattern evaluates to the correct crochet graph.		
`hooklib::pattern::tests::test_flat`	Pass	0.000828
`hooklib::pattern::tests::test_sphere`	Pass	0.0022489
`hooklib::pattern::tests::test_joined_rounds`	Pass	0.0043512
`hooklib::pattern::tests::test_spiral_rounds`	Pass	0.0052245
Tests that various example patterns evaluate to the correct crochet graph.		
`hooklib::pattern::tests::test_triangulated`	Pass	0.0205736
Tests that the triangulation process runs without errors.		
`hooklib::script::tests::test_script`	Pass	0.003869
Tests that a script evaluates to the correct crochet graph.		
`hooklib::script::tests::test_all_examples`	Pass	0.0249559
Tests that various example scripts evaluate with no errors.		
`polyhook::render::pattern_model::tests::test_runtime`	Pass	5.2666012
Tests that the total runtime of the whole process is less than 30 seconds for a graph of 2401 (49^2) nodes.		
`polyhook::render::pattern_model::tests::test_analyze_runtime`	Pass	38.5607979
Analyzes the runtime as nodes increase, used to produce the graph in Figure 8.5		
`sgd::tests::test_sgd`	Pass	0.0874047
`sgd::tests::test_sgd_size`	Pass	0.503257
Tests that the SGD process runs without errors.		

Table C.1: Table of test runtimes and results, along with descriptions of testing purpose.